

Chapter 1. Basic for CSE



Lee Youngkon



Why Do We Need Object–Oriented Programming?

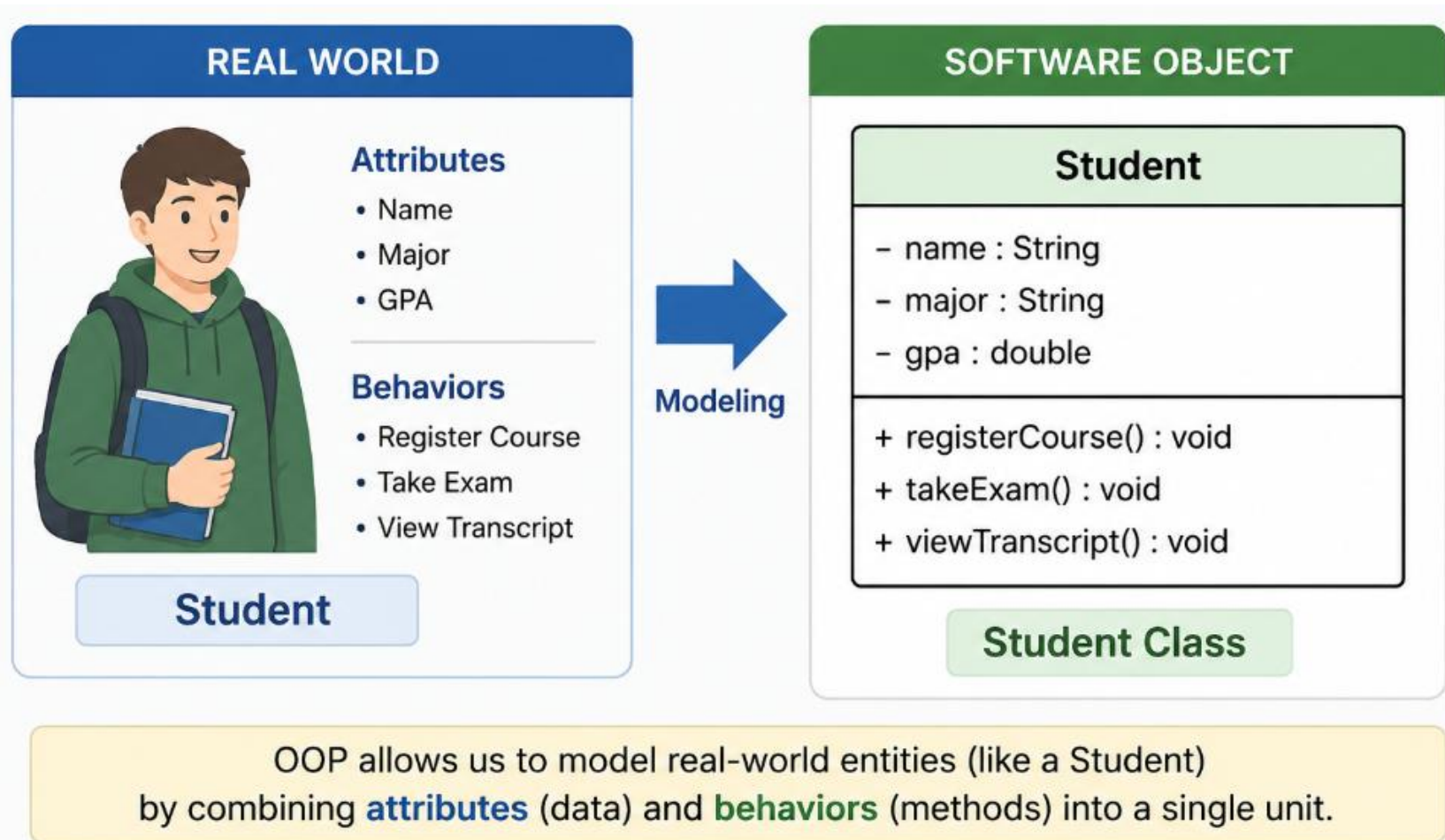
- ✓ Software is Becoming More Complex
 - Modern systems contain:
 - Thousands of functions
 - Millions of lines of code
 - Multiple developers working together
- ✓ Examples:
 - ERP Systems
 - Banking Systems
 - E–Commerce Platforms
 - Social Media Applications
- ✓ Problem
 - As software grows, maintaining procedural code becomes difficult.

Procedural Programming

- ✓ Focuses on:
 - Functions
 - Procedures
 - Sequential execution

Challenges

- ✓ Data and functions are separate
- ✓ Difficult maintenance
- ✓ Code duplication
- ✓ Poor scalability
- ✓ Hard to collaborate in large projects



OOP Definition

- ✓ Object–Oriented Programming (OOP) is a programming paradigm that organizes software around objects.
- ✓ An object contains:
 - Data (Attributes)
 - Behavior (Methods)
- ✓ OOP combines data and behavior into a single unit.

- ✓ Maintainability
 - Easier to modify and manage code
- ✓ Reusability
 - Reuse existing classes
- ✓ Scalability
 - Easy to add new features
- ✓ Collaboration
 - Supports team-based development
- ✓ Real-World Modeling
 - Software structure matches real-world concepts

What is a Class?

- ✓ A Class is a blueprint or template used to create objects.
- ✓ It defines:
 - Attributes (Data)
 - Methods (Behavior)

```
class Student {  
    String name;  
    int age;  
  
    void study() {  
        System.out.println("Studying...");  
    }  
}
```

What is an Object?

- ✓ Object = Instance of a Class
- ✓ An object is a real entity created from a class.
- ✓ Creating Objects

```
Student s1 = new Student();  
Student s2 = new Student();
```

Class	Object
Blueprint	Actual Instance
Definition	Real Entity
No Memory Allocated	Memory Allocated
One Class	Many Objects

What is an Object?

- ✓ Object = Instance of a Class
- ✓ An object is a real entity created from a class.
- ✓ Creating Objects

```
Student s1 = new Student();  
Student s2 = new Student();
```

Class	Object
Blueprint	Actual Instance
Definition	Real Entity
No Memory Allocated	Memory Allocated
One Class	Many Objects

	Class	Object
 Meaning	Blueprint or template	Actual instance of a class
 Definition	Defines attributes and methods	Has actual values for attributes
 Memory	No memory allocated	Memory is allocated
 Quantity	One class	Many objects can be created
 Example	Student (class)	s1, s2, s3 (objects)

Example

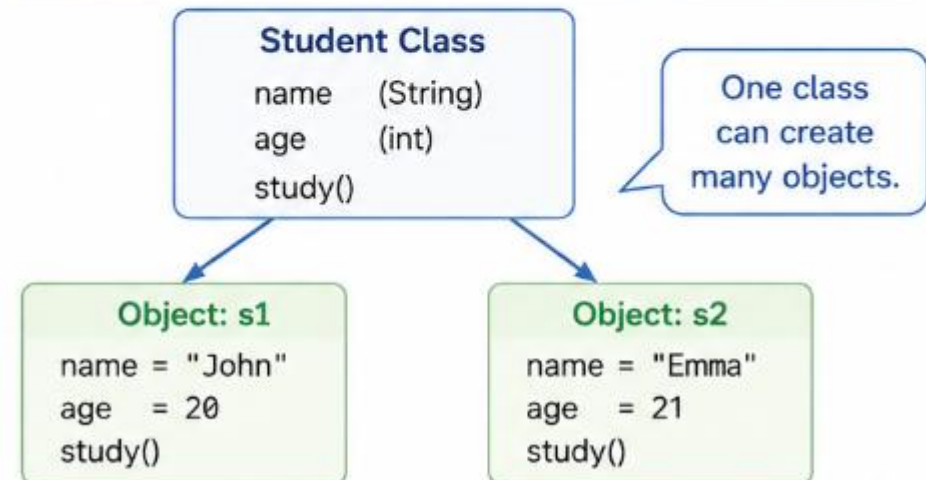
```

class Student {
    String name;
    int age;

    void study() {
        System.out.println("Studying...");
    }
}

// Creating objects
Student s1 = new Student();
Student s2 = new Student();

```



Class is the **blueprint**. Object is the **real thing**.

Object State and Behavior

✓ Every object has:

- State (Attributes)

name = "John"

age = 20

- Behavior (Methods)

study()

takeExam()

```
Student s1 = new Student();
```

```
s1.name = "John";
```

```
s1.age = 20;
```

```
s1.study();
```

Why Use Classes and Objects?

✓ Every object has:

- State (Attributes)

name = "John"

age = 20

- Behavior (Methods)

study()

takeExam()

```
Student s1 = new Student();
```

```
s1.name = "John";
```

```
s1.age = 20;
```

```
s1.study();
```

Concept

- ✓ Component-Based Design (CBD) is a software development approach that builds applications by assembling reusable and independent software components.
- ✓ A component is a self-contained unit that provides specific functionality through well-defined interfaces while hiding its internal implementation.
- ✓ Key Characteristics
 - Reusability
 - Modularity
 - Encapsulation
 - Replaceability
 - Maintainability
 - Loose Coupling

characteristics

1. Reusability



Components are designed for reuse in multiple applications, reducing development time and cost.

2. Modularity



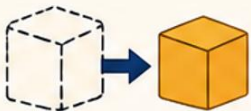
The system is divided into independent, self-contained modules (components).

3. Encapsulation



Each component hides its internal details and exposes only necessary interfaces.

4. Replaceability



Components can be replaced with another compatible component without affecting the whole system.

5. Maintainability



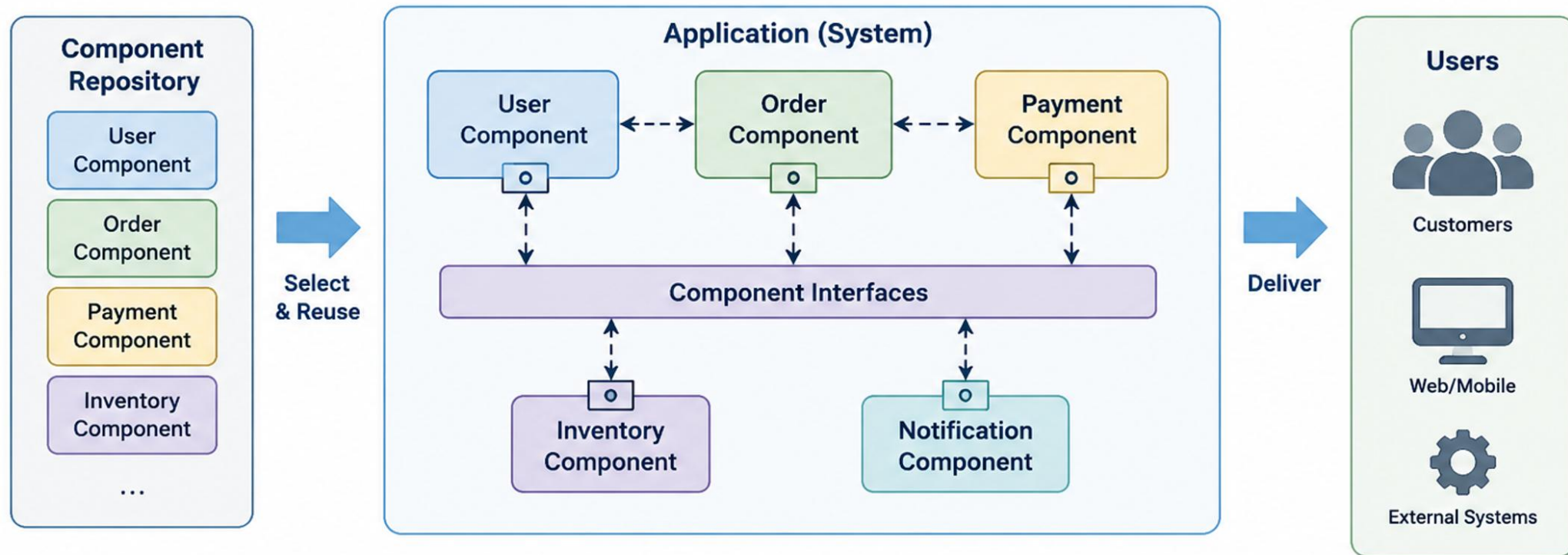
Components are easy to update, test, and maintain independently, improving productivity.

6. Loose Coupling



Components interact through well-defined interfaces with minimal dependencies.

CBD Architecture



CBD Key Elements



Component

Reusable, self-contained unit providing specific functionality



Interface

Defines the services provided by a component



Connector

Enables communication and interaction between components



Repository

Stores and manages reusable components



Assembly

Integrates selected components into the system

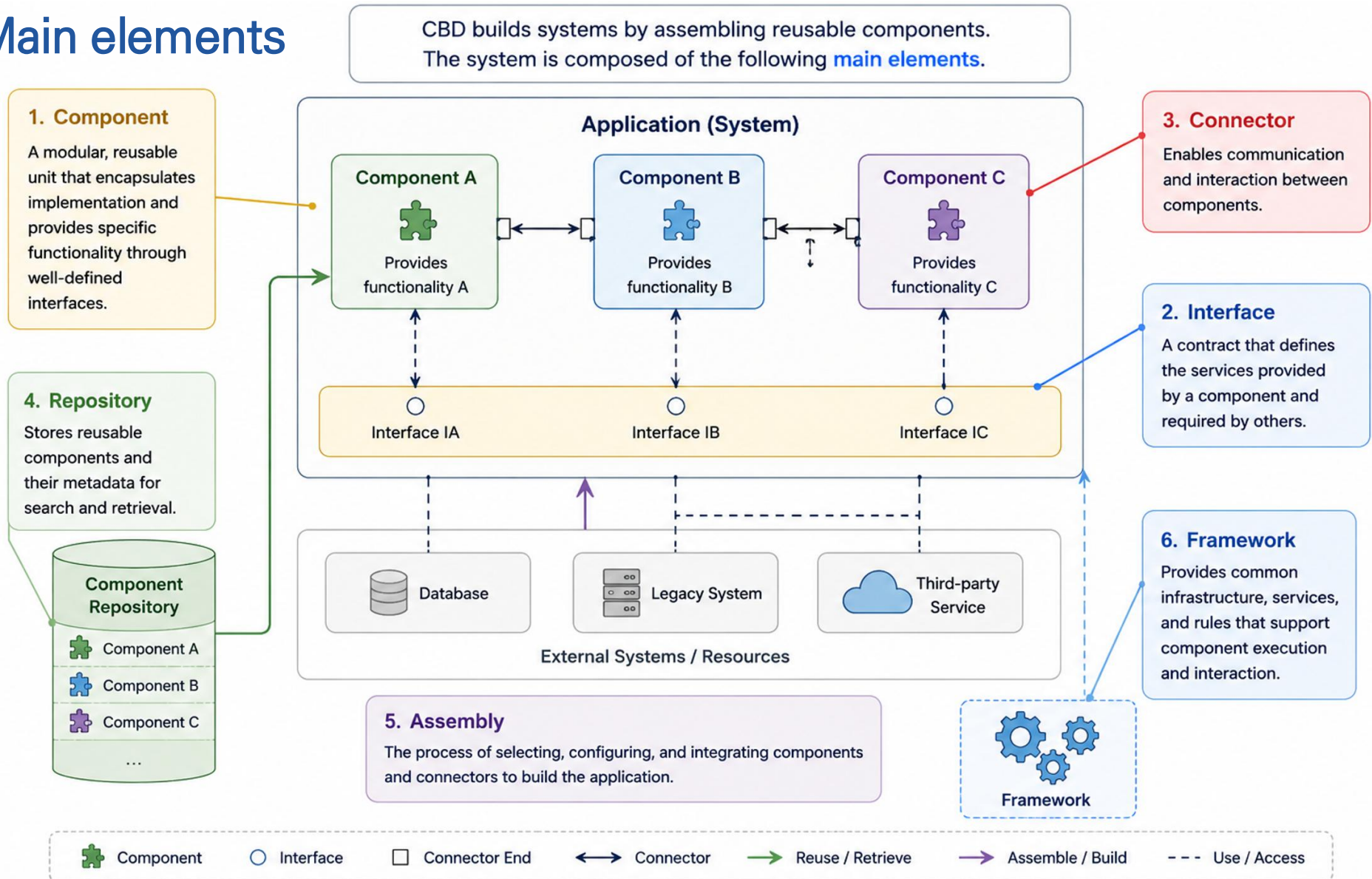


Framework

Provides infrastructure and common services for components

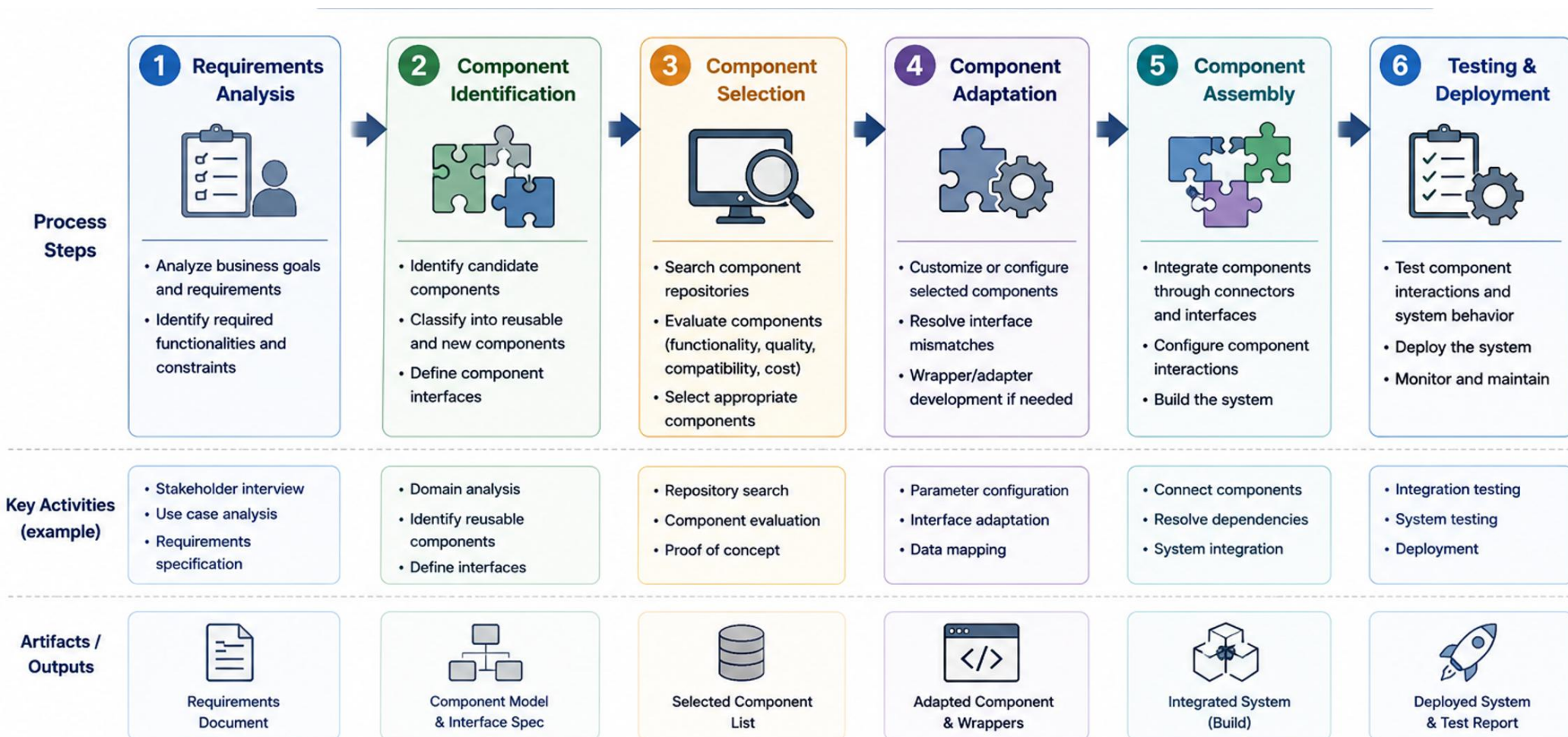
Component-Based Design (CBD)

Main elements



Component-Based Design (CBD)

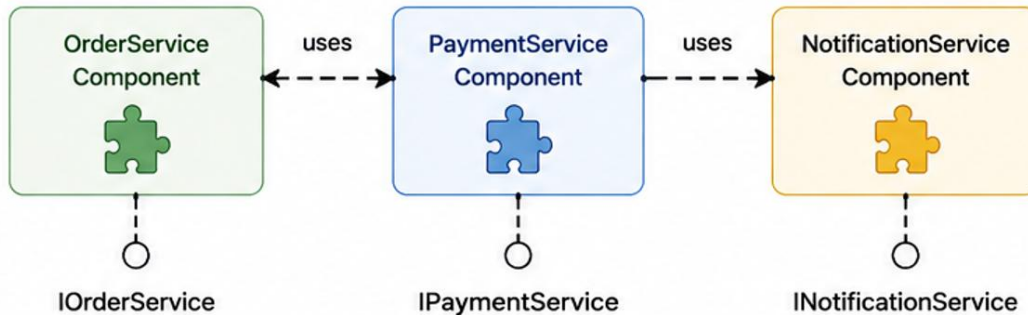
CBD development process



Component-Based Design (CBD)

CBD example

Component Structure



1. Interface (Contract)

```

1  public interface IPaymentService {
2
3      boolean processPayment(double amount);
4
5  }
6
  
```

2. Component Implementation

```

1  public class CreditCardPayment implements IPaymentService {
2
3      @Override
4      public boolean processPayment(double amount) {
5          System.out.println("Payment completed: " + amount);
6          return true;
7      }
8  }
  
```

3. Component Usage (Client)

```

1  public class OrderService {
2      private IPaymentService paymentService;
3
4      public OrderService(IPaymentService paymentService) {
5          this.paymentService = paymentService;
6      }
7
8      public void placeOrder(double amount) {
9          boolean result = paymentService.processPayment(amount);
10         System.out.println("Order result: " + result);
11     }
12 }
  
```

4. How to Use

```

1  IPaymentService paymentService = new CreditCardPayment();
2  OrderService orderService = new OrderService(paymentService);
3  orderService.placeOrder(150.0);
4
  
```



Key Points

- Components provide functionality through Interfaces.
- Clients depend on Interfaces, not on concrete implementations.
- Implementation can be easily replaced without changing the client.

CBD advantage and disadvantage

Advantages	Disadvantages
Faster development	Component compatibility issues
Reusability	Integration complexity
Easier maintenance	Version management challenges
Reduced development cost	Vendor dependency
Improved quality	Performance overhead

Concept

- ✓ An Entity Relationship Diagram (ERD) is a visual modeling technique used to represent the structure of data within a system.
- ✓ It illustrates entities, attributes, and relationships, providing a blueprint for database design.
- ✓ ERDs help analysts, designers, and developers understand how data is organized and connected before implementing a database.

Benefits

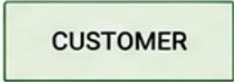
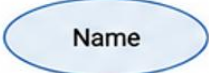
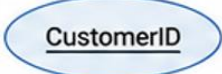
- ✓ Understand business data requirements
- ✓ Design databases systematically
- ✓ Reduce data redundancy
- ✓ Improve data consistency and integrity
- ✓ Facilitate communication between stakeholders
- ✓ Serve as documentation for future maintenance

Main Components

Component	Description	Example
Entity	A real-world object or concept	Customer, Product
Attribute	Property of an entity	CustomerID, Name
Primary Key (PK)	Unique identifier	CustomerID
Foreign Key (FK)	Reference to another entity	CustomerID in Order
Relationship	Association between entities	Customer places Order
Cardinality	Number of participating instances	1:1, 1:N, M:N

Entity Relational Diagram(ERD)

Main component

1. ERD NOTATION			
Symbol	Name	Description	Example
	Entity	A real-world object or concept.	
	Attribute	A property or characteristic of an entity.	
	Key Attribute (Primary Key)	An attribute that uniquely identifies an entity.	
	Derived Attribute	An attribute whose value is derived from other data.	
	Relationship	An association between two or more entities.	
	Connection	A line that connects entities and relationships.	
	Multivalued Attribute	An attribute that can have multiple values.	
	Weak Entity	An entity that cannot be uniquely identified without another entity.	
	Identifying Relationship	A relationship that identifies a weak entity.	

Entity Relational Diagram(ERD)

Cardinality

2. CARDINALITY (RELATIONSHIP TYPES)				
Type	Notation (Chen)	Notation (Crow's Foot)	Example	Description
One-to-One (1:1)				Each entity instance on one side is associated with at most one instance on the other side.
One-to-Many (1:N)				One entity instance on the "1" side can be associated with many instances on the "N" side.
Many-to-One (N:1)				Many entity instances on the "N" side are associated with one instance on the "1" side.
Many-to-Many (M:N)				Many instances on one side can be associated with many instances on the other side.



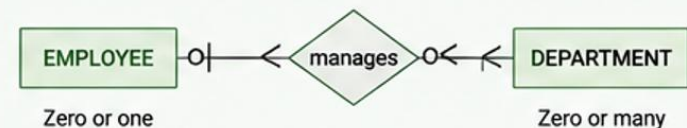
Notes

- Cardinality defines the number of instances that can participate in a relationship.
- Chen notation uses diamonds and numbers (1, N, M).
- Crow's Foot notation uses symbols: | (one), < (many), > (many), and combinations.

Crow's Foot Symbols

- | One and only one
- < Many
- |< One or many (optional many)
- o| Zero or one (optional one)
- o< Zero or many (optional many)

Example with Optionality



Create Process

1

Identify Entities

Identify the main entities (objects or concepts) that exist in the system.

Example

CUSTOMER

ORDER

PRODUCT

PAYMENT

2

Identify Attributes

Determine the attributes (properties) for each entity.

Example

CUSTOMER

- CustomerID (PK)
- Name
- Email
- Phone

ORDER

- OrderID (PK)
- OrderDate
- CustomerID (FK)

3

Define Primary Keys

Choose a primary key (PK) for each entity to uniquely identify records.

Example

CUSTOMER

- CustomerID (PK)
- Name
- Email
- Phone

ORDER

- OrderID (PK)
- OrderDate
- CustomerID (FK)

4

Identify Relationships

Identify the relationships between entities and give them meaningful names.

Example

CUSTOMER

places

ORDER

contains

PRODUCT

5

Define Cardinalities

Define the cardinality (1:1, 1:N, N:1, M:N) for each relationship.

Example

CUSTOMER

1

places

N

ORDER

1

contains

N

PRODUCT

6

Review & Refine

Review the ERD for accuracy, remove redundancies, and improve the model.

Example

CUSTOMER

- CustomerID (PK)
- Name
- Email
- Phone

1

places

N

ORDER

- OrderID (PK)
- OrderDate
- CustomerID (FK)

1

contains

N

PRODUCT

- ProductID (PK)
- ProductName
- Price

Concepts

- ✓ UML (Unified Modeling Language) is a standardized visual modeling language used to analyze, design, and document software systems.
- ✓ UML helps stakeholders understand system structure, behavior, and interactions before implementation.
- ✓ Purpose
 - Visualize system architecture
 - Analyze requirements
 - Design software solutions
 - Improve communication among stakeholders
 - Document software systems

Key Characteristics

1

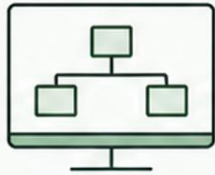
Standardized



Uses internationally recognized notation and semantics.

2

Visual



Represents system structure and behavior graphically for easy understanding.

3

Object-Oriented



Designed based on object-oriented concepts such as classes, objects, inheritance, and polymorphism.

4

Platform Independent



Can be applied regardless of programming language or development platform.

5

Comprehensive



Provides a wide range of diagrams to model structure, behavior, and interactions.

6

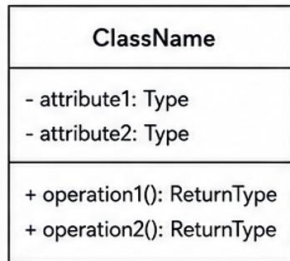
Extensible



Can be extended with new mechanisms and custom profiles for specific domains.

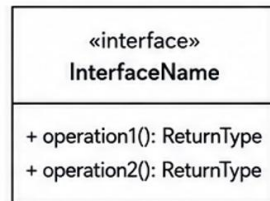
Main components

1. Class



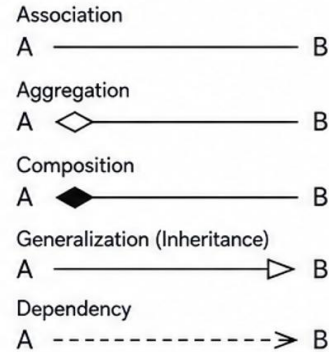
A basic building block that represents the structure of a system, including attributes and operations.

2. Interface



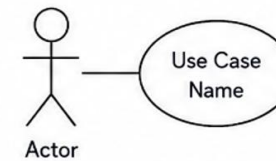
A contract that defines a set of operations (behaviors) that a class must implement.

3. Relationship



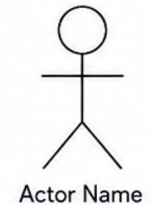
Relationships show how elements are connected.

4. Use Case



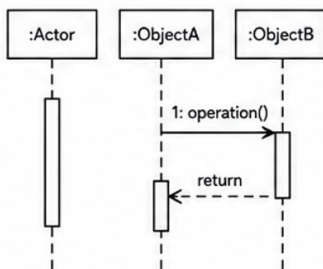
Represents a functionality (or service) provided by the system from the user's perspective.

5. Actor



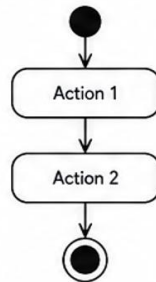
An external entity that interacts with the system, such as a person, another system, or external entity.

6. Sequence Diagram



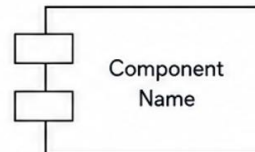
Shows the time order of messages exchanged between objects.

7. Activity Diagram



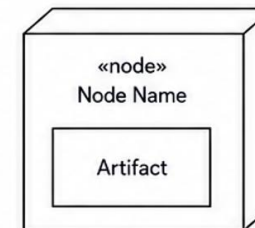
Models the workflow of activities and control flow within a process.

8. Component Diagram



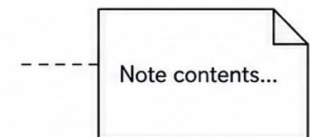
Represents the physical and software components of a system.

9. Deployment Diagram



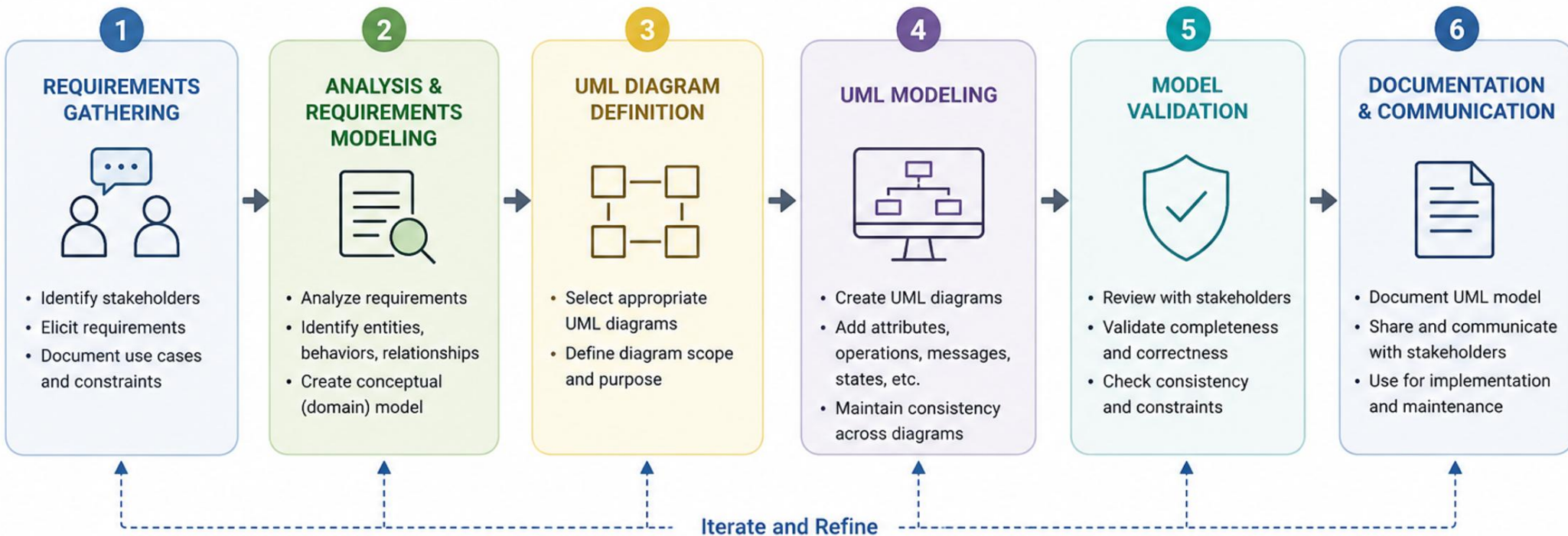
Shows the physical deployment architecture, nodes, and artifacts.

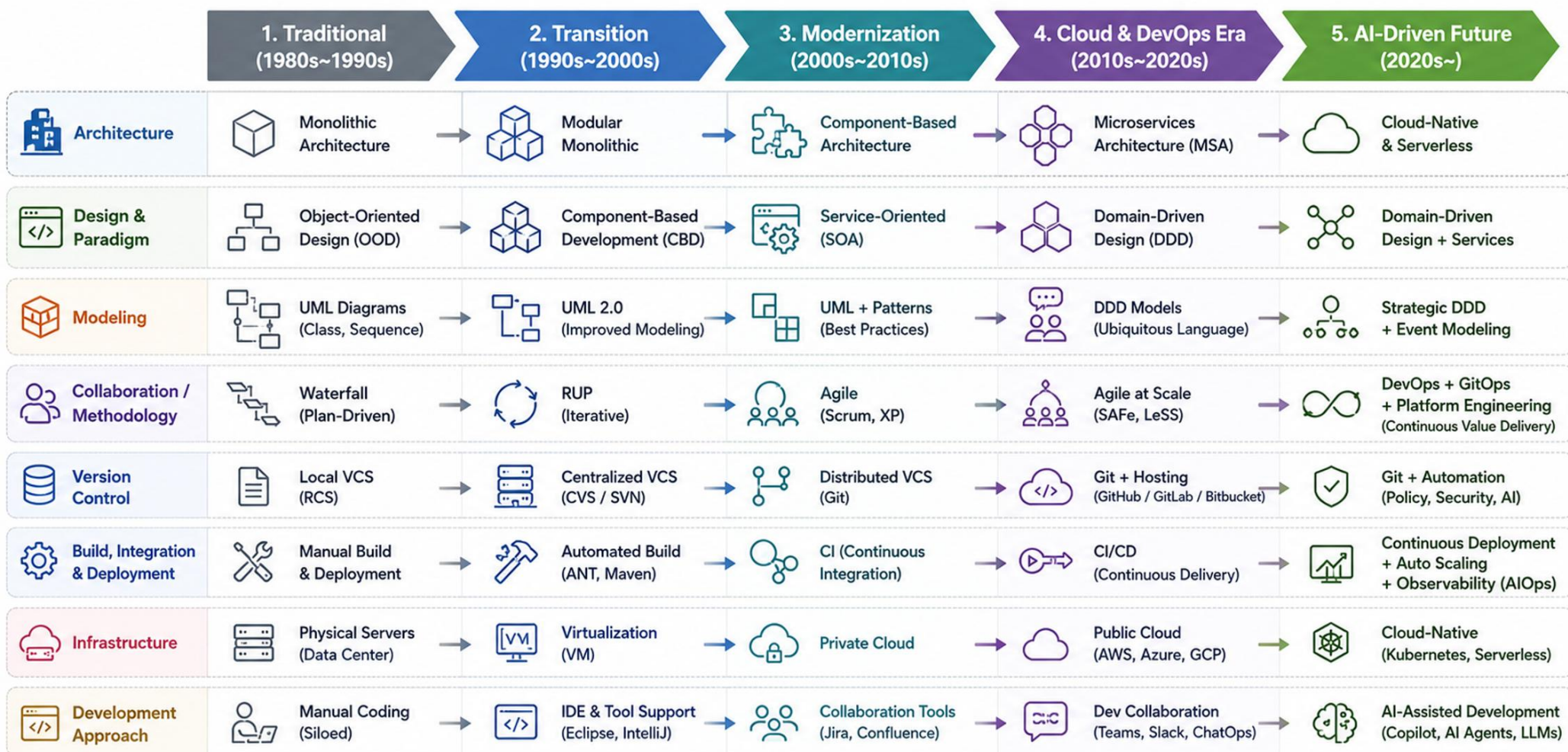
10. Note



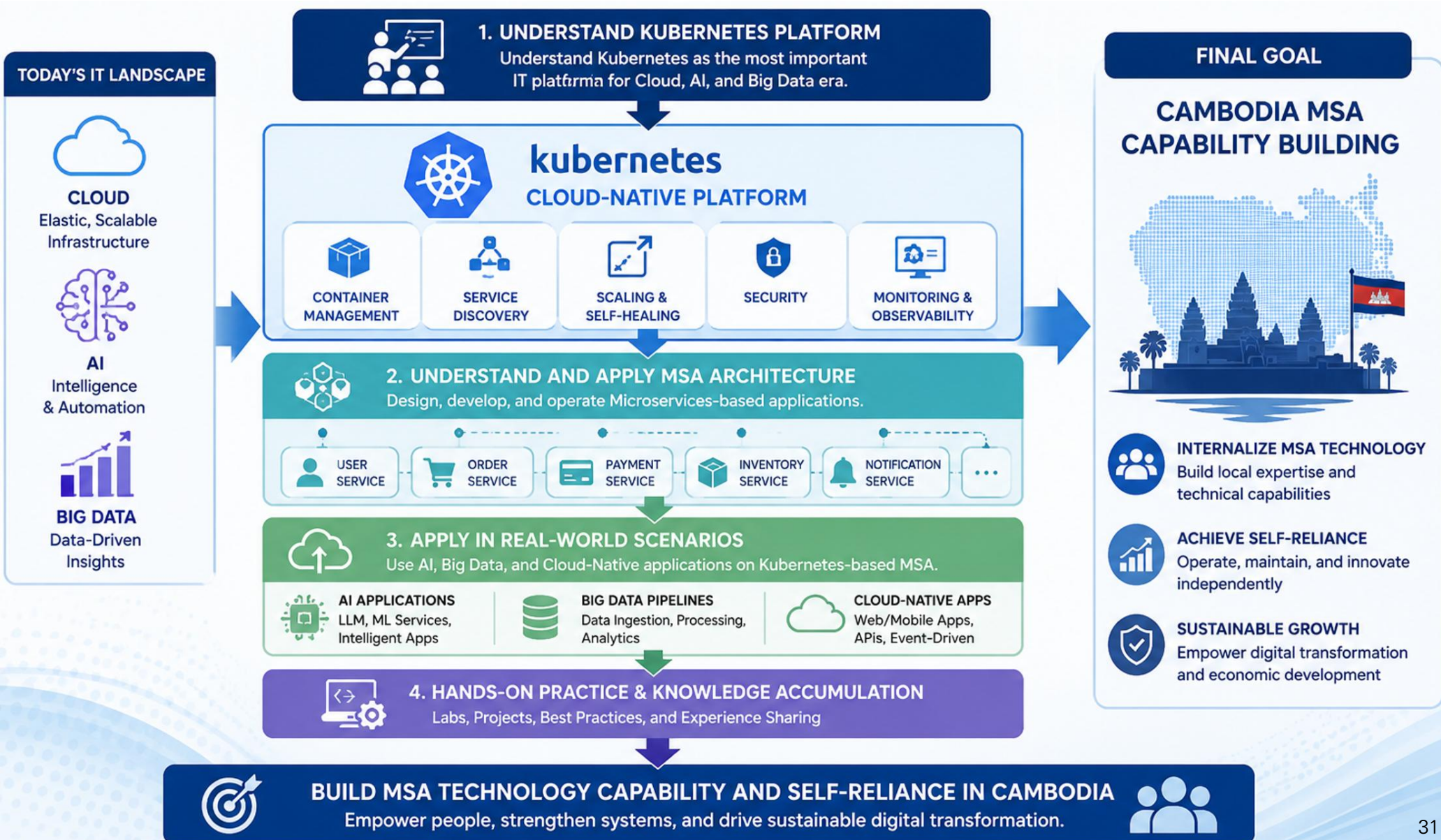
Provides comments or constraints to describe elements or diagrams.

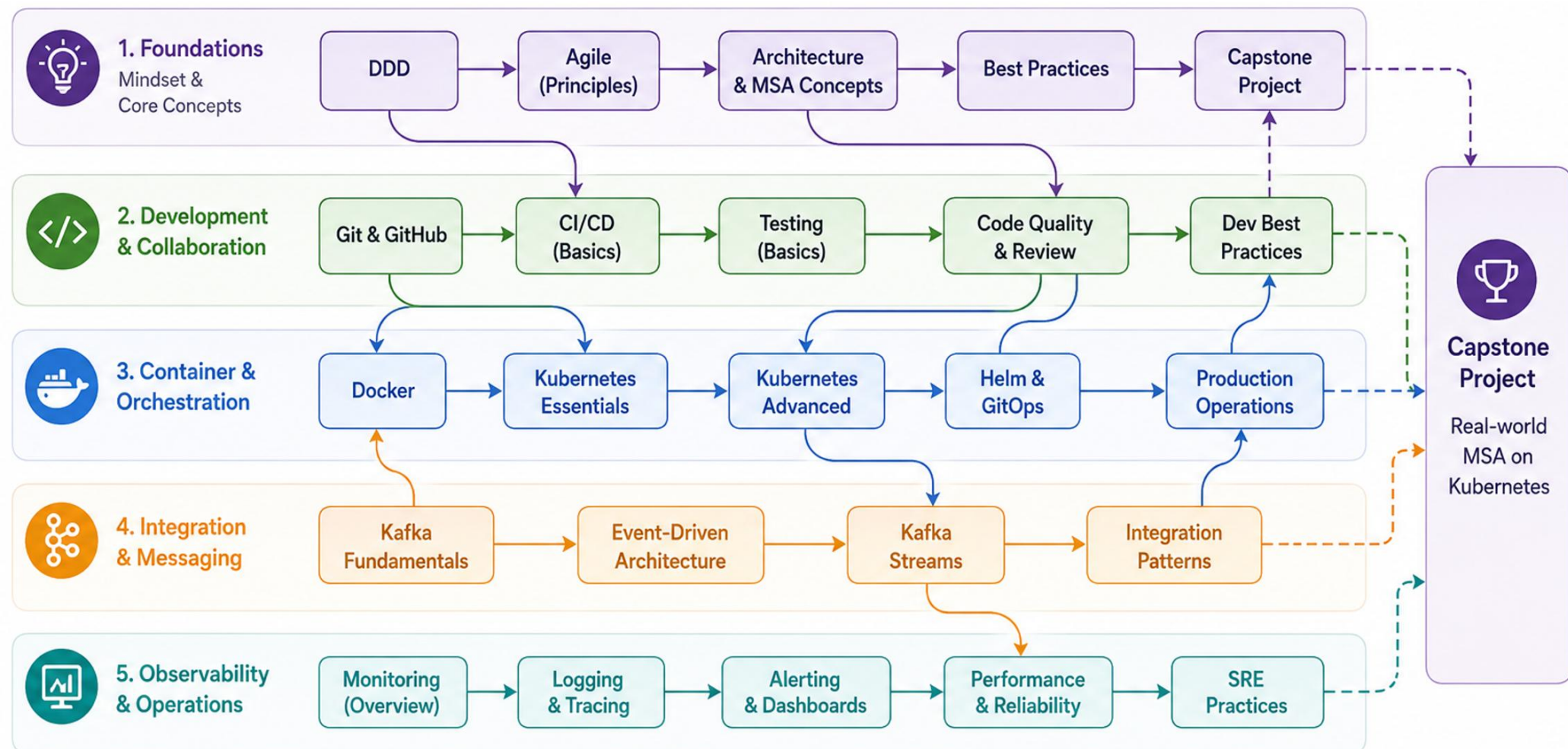
Create process





Purpose of Training





Prerequisites

- ✓ Create a Docker Hub account: <https://hub.docker.com>
- ✓ Create an Atlassian account (for Jira): <https://id.atlassian.com>
- ✓ Create a GitHub account: <https://github.com>
- ✓ Verify your email address for each account before the lab.
- ✓ Keep your account credentials available for hands-on exercises.

Process for Java 17

- ① Download JDK 17 from the official OpenJDK or Oracle website.
<https://www.oracle.com/java/technologies/javase/jdk17-archive-downloads.html>
- ② Run the installer and follow the installation wizard.
- ③ Set the JAVA_HOME environment variable to the JDK installation directory.
- ④ Add %JAVA_HOME%\bin to the PATH environment variable.
- ⑤ Verify the installation by running `java -version` and `javac -version` in a terminal.

Process for VS code extention installation

- ① Open Visual Studio Code.
- ② Click the Extensions icon on the Activity Bar (or press Ctrl + Shift + X).
- ③ Search for the required extension (e.g., Extension Pack for Java or Spring Boot Extension Pack).
- ④ Click Install to install the extension.
- ⑤ Restart Visual Studio Code if prompted.
- ⑥ Verify the installation by checking that the extension appears in the Installed list.

Preparation for Practice in VS Code

Extension	Purpose
Extension Pack for Java	Provides a complete Java development environment, including language support, debugging, testing, Maven integration, and project management.
Maven for Java	Provides Maven project management, dependency management, build execution, and lifecycle support.
Spring Boot Extension Pack	A collection of extensions for Spring Boot development, including Spring Initializr, Spring Boot Tools, and Dashboard support.
Spring Boot Tools	Provides Spring Boot project support, application launch, live information, and development utilities.
Spring Initializr Java Support	Enables creating new Spring Boot projects directly from Visual Studio Code using Spring Initializr.

Process for MySQL 8 installation

- ① Download MySQL Installer from the official MySQL website.
<https://dev.mysql.com/downloads/windows/installer/8.0.html>
- ② Run the installer and select MySQL Server 8.0. Choose the installation type (e.g., Developer Default or Custom).
- ③ Configure the MySQL Server, including the root password and authentication method.
- ④ Complete the installation and start the MySQL service.
- ⑤ Verify the installation by connecting with MySQL Workbench or running `mysql --version` in a terminal.

Process for HeidiSQL installation

- ① Download HeidiSQL from the official HeidiSQL website.
<https://www.heidisql.com/download.php>
- ② Run the installer and follow the installation wizard.
- ③ Launch HeidiSQL after the installation is complete.
- ④ Create a new session and enter the MySQL server information (host, port, username, and password).
- ⑤ Click Open to connect to the MySQL server.
- ⑥ Verify the connection and start managing databases.

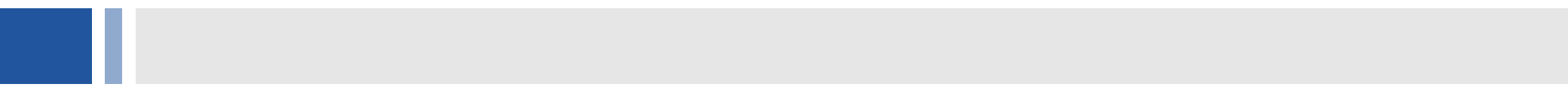
Process for Git installation

- ① Download Git from the official Git website.
<https://git-scm.com/install/windows>
- ② Run the Git installer.
- ③ Select the recommended installation options.
- ④ Complete the installation.
- ⑤ Verify the installation by running `git --version` in a terminal.
- ⑥ Configure Git with your username and email.

```
git config --global user.name "Your Name"  
git config --global user.email "your.email@example.com"
```

Process for Practice

- ① Sign in to GitHub.
- ② Open github.com/leeyoungkon.
- ③ Select the target repository.
- ④ Click Fork to create your own copy.
- ⑤ Copy the repository URL.
- ⑥ Open Visual Studio Code.
- ⑦ Clone the forked repository (Git: Clone).
- ⑧ Open the cloned project.
- ⑨ Start coding and commit your changes.



Thank You